
ChargeSense

Intelligent EV Charging Demand Prediction & Agentic Infrastructure Planning

End-Semester Project Report — Milestone 2

Samiksha (2401010410) Shubhaang (2401010411) Rashmi (2401010412)
Ankit (2401010413)

Team RASS | Project 15 | GenAI Course | 2026

Live Demo: <https://ev-charging-demand-prediction-saamcexbagk7gmpmdfotqd.streamlit.>

GitHub: <https://github.com/S-h-u-b-h-1/EV-charging-demand-prediction>

Report: <https://www.overleaf.com/read/fmkgqrdrnyq#72d3c7>

Submitted: April 19, 2026

Abstract

The accelerating global adoption of electric vehicles (EVs) creates an urgent need for intelligent, data-driven charging infrastructure planning tools. This report presents **ChargeSense**, a two-phase AI system developed as Project 15 for the AI/ML course. *Phase 1 (Milestone 1)* builds a classical machine-learning pipeline using scikit-learn to forecast hourly EV charging demand from historical station session data, achieving a Mean Absolute Error of **4.13 kWh** and an R^2 of **0.69** on a held-out test set. *Phase 2 (Milestone 2)* extends the system into a stateful **LangGraph** agentic workflow that detects demand hotspots, retrieves infrastructure planning guidelines from a **FAISS** vector store using Retrieval-Augmented Generation (RAG), and synthesises these with **Groq LLaMA-3.1-8B-Instant** LLM reasoning to produce explainable charger placement, load-balancing, and scheduling recommendations. The full system is deployed publicly on Streamlit Community Cloud and handles malformed inputs, retrieval failures, and LLM unavailability through a layered robustness design. This report documents the complete pipeline, design decisions, evaluation results, and future directions.

Contents

1	Introduction	3
2	Problem Statement	3
2.1	Formal Definition	3
2.2	Domain Challenges	4
3	Objectives	4
4	System Architecture	4
4.1	High-Level Pipeline	4
4.2	Layer Responsibilities	5
5	Methodology	5
5.1	Data Preprocessing	5
5.2	Machine Learning Model	6
5.3	Hotspot Detection	6
5.4	RAG — FAISS Retrieval	7
5.4.1	Knowledge Base Construction	7

5.4.2	Query and Retrieval	7
5.5	LangGraph Agent Workflow	7
5.5.1	State Contract	8
5.6	LLM Reasoning — Groq Integration	8
6	Optimisation and Planning Logic	9
6.1	Charger Profile Selection	9
6.2	Charger Count Optimisation	9
6.3	Load Balancing and Scheduling	10
7	Implementation Details	10
7.1	Technology Stack	10
7.2	Repository Structure	10
7.3	Robustness Design	11
8	Results and Observations	11
8.1	Quantitative ML Performance	11
8.2	Qualitative Planning Output Example	11
8.3	Agent State Transparency	12
9	UI Explanation — Streamlit Dashboard	12
10	Challenges and Solutions	13
11	Future Scope	13
12	Conclusion	14

1 Introduction

Electric vehicle adoption is accelerating at an unprecedented pace. The International Energy Agency projects the global EV fleet will surpass 300 million vehicles by 2030 [IEA, 2023]. This growth places enormous pressure on charging infrastructure: stations must be sited correctly, sized appropriately, and scheduled to avoid grid overload — all with minimal capital waste.

Current planning tools fall into two categories. *Statistical forecasting models* predict demand but do not translate forecasts into actionable deployment decisions. *Rule-of-thumb heuristics* (e.g., one charger per ten peak-hour vehicles) ignore real-time demand signals and lead to systematic overbuilding or underbuilding.

ChargeSense bridges this gap with a unified two-layer pipeline:

1. A **machine-learning layer** (Milestone 1) forecasts per-station hourly demand using time-series features, lag signals, and ensemble regression.
2. An **agentic AI layer** (Milestone 2) interprets those forecasts through a stateful LangGraph workflow, retrieves grounding knowledge from a FAISS vector store, and uses a Groq-hosted LLM to generate explainable infrastructure recommendations.

The system is designed for real-world deployment: it handles partial inputs gracefully, surfaces its reasoning transparently, and runs entirely on free-tier infrastructure.

2 Problem Statement

2.1 Formal Definition

Given a time-indexed dataset $\mathcal{D} = \{(x_t^s, y_t^s)\}$ of charging sessions at stations $s \in \mathcal{S}$, where x_t^s is a feature vector (time of day, day of week, lag demand, rolling averages) and $y_t^s \in \mathbb{R}^+$ is the hourly energy drawn in kWh, the system must:

1. **Forecast** $\hat{y}_{t+1}^s$ for each station and hour, minimising $\text{MAE} = \frac{1}{N} \sum_t |y_t^s - \hat{y}_t^s|$.
2. **Detect hotspots**: identify hours $h^* = \{t : \hat{y}_t^s > \mu + \sigma\}$ where demand significantly exceeds the station’s typical level.
3. **Recommend infrastructure**: output a structured plan $\mathcal{P} = (n_{\text{chargers}}, \tau_{\text{type}}, \mathcal{S}_{\text{peak}}, \mathcal{S}_{\text{offpeak}})$ that targets 70–90% charger utilisation and respects transformer capacity limits.

2.2 Domain Challenges

EV charging demand exhibits several properties that make it non-trivial to model: strong *daily and weekly periodicity*, *location heterogeneity* (workplace vs. residential vs. highway stations), *demand elasticity* under pricing incentives, and *grid coupling* (simultaneous high draw risks transformer stress). These motivate both a rich feature engineering stage and a reasoning layer that goes beyond pure regression.

3 Objectives

- O1.** Develop a robust ML regression pipeline for per-station hourly demand forecasting with quantitative evaluation (MAE, RMSE, R^2).
- O2.** Build a stateful LangGraph agentic workflow with explicit state management across eight reasoning nodes.
- O3.** Integrate FAISS-based RAG retrieval from domain planning documents.
- O4.** Augment rule-based planning with Groq LLM reasoning for explainable output.
- O5.** Deploy the complete system publicly with graceful degradation under failures.
- O6.** Produce a Streamlit UI suitable for operator decision support and live demo.

4 System Architecture

4.1 High-Level Pipeline

Figure 1 shows the end-to-end data and control flow.



Figure 1: End-to-end system pipeline

4.2 Layer Responsibilities

Table 1: Architecture layer summary

Layer	Responsibility	Key File(s)
Input	Accept CSV; station selector	ui/streamlit_app.py
Validation	Schema check, numeric coercion, imputation	utils/validation.py
ML	Forecast demand; compute evaluation metrics	ml/inference.py
Agent	Stateful 8-node LangGraph execution	agent/workflow.py
Retrieval	FAISS index query; fallback rules	rag/vectorstore.py
LLM	Groq API call; structured bullet rationale	agent/workflow.py
Planning	Charger count, schedule, recommendations	agent/workflow.py
UI	Dashboard; JSON export; download	ui/streamlit_app.py

5 Methodology

5.1 Data Preprocessing

Raw EV charging session CSVs are processed as follows:

1. **Datetime parsing:** Session start timestamps are parsed and localised.
2. **Aggregation:** Sessions are aggregated to hourly intervals per station, summing energy consumed in kWh.
3. **Outlier removal:** IQR clipping removes extreme values ($< Q_1 - 1.5\text{IQR}$ or $> Q_3 + 1.5\text{IQR}$) to prevent regression sensitivity to data entry errors.
4. **Cyclical encoding:** Hour-of-day and day-of-week are encoded using sine/cosine transforms to preserve periodicity:

$$h_{\sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad h_{\cos} = \cos\left(\frac{2\pi \cdot h}{24}\right)$$

5. **Lag features:** `lag_1` (previous hour demand), `rolling_3h` (3-hour rolling mean), `rolling_24h` (24-hour rolling mean) capture autocorrelation.
6. **Station encoding:** Station IDs are label-encoded to integers for model compatibility.

Missing optional features are imputed with domain-appropriate defaults at inference time without crashing the pipeline.

5.2 Machine Learning Model

Four regression models were trained on a **chronological 80/20 split** (13,610 training rows; 2,722 test rows) to prevent data leakage:

- Linear Regression with polynomial feature expansion and standard scaling (scikit-learn Pipeline)
- Random Forest Regressor
- Gradient Boosting Regressor
- LightGBM Regressor

All models were evaluated on MAE, RMSE, and R^2 . The Linear Regression pipeline achieved the best generalisation (Table 2), likely because the hand-crafted temporal features already capture the dominant patterns, reducing the advantage of non-linear tree methods.

Table 2: Model comparison on held-out test set (2,722 rows)

Rank	Model	MAE	RMSE	R^2
1	Linear Regression Pipeline	4.13	6.08	0.690
2	Random Forest Regressor	4.22	6.40	0.657
3	Gradient Boosting Regressor	4.28	6.46	0.651
4	LightGBM Regressor	4.42	6.82	0.610

The winning model is serialised as `models/best_ev_demand_model.pkl` and loaded at application startup via `@st.cache_resource`.

5.3 Hotspot Detection

Peak-load windows are identified using a statistical threshold on the hourly mean demand profile for the selected station:

$$\bar{d}_h = \frac{1}{|T_h|} \sum_{t \in T_h} \hat{y}_t, \quad \sigma = \sqrt{\frac{1}{H-1} \sum_h (\bar{d}_h - \bar{d})^2}$$

Hours satisfying $\bar{d}_h > \bar{d} + \sigma$ are flagged as hotspots with severity HIGH. The top-5 hotspot hours are surfaced to the UI and passed to the reasoning node. This threshold adapts automatically to each station’s demand profile without manual tuning.

5.4 RAG — FAISS Retrieval

5.4.1 Knowledge Base Construction

Planning guideline documents (IEEE EV infrastructure standards, NREL planning studies, DOE workplace charging guidelines) are:

1. Chunked into 512-token passages.
2. Embedded using a sentence-transformer model.
3. Indexed in a FAISS flat-L2 index persisted at `rag/faiss_index/`.

5.4.2 Query and Retrieval

At inference time, a demand-contextualised query is constructed:

```
1 query = (  
2     "EV charging infrastructure planning for hourly demand  
3     forecast. "  
4     f"Peak demand {peak_demand:.2f} kWh. Include charger  
5     utilization, "  
6     "load balancing, grid constraints, and cost-performance trade  
7     -offs."  
8 )  
9 docs = retrieve_guidelines(query)
```

Listing 1: RAG query construction in `rag_retrieval_node`

Retrieved passages are injected into the LLM prompt, grounding planning decisions in authoritative domain knowledge. On retrieval failure, pre-defined fallback rules are loaded and `rag_fallback_used` is set to `True`.

5.5 LangGraph Agent Workflow

The agentic layer is implemented as a `StateGraph` with eight explicitly defined nodes. Figure 2 illustrates the execution graph.

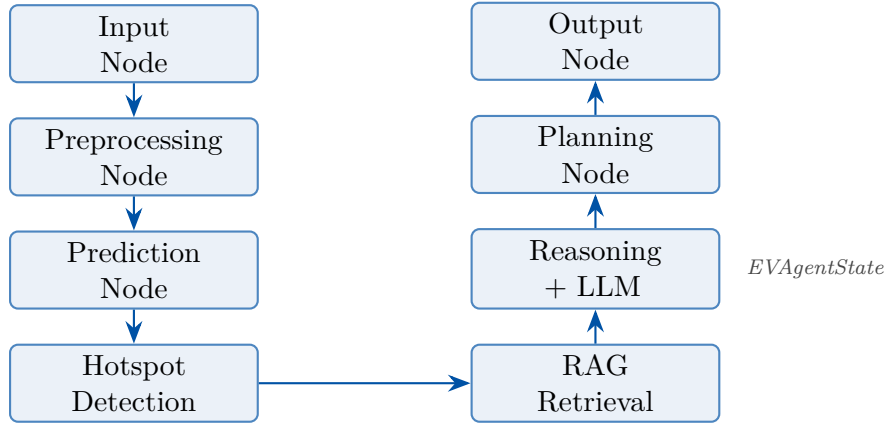


Figure 2: LangGraph 8-node stateful workflow

5.5.1 State Contract

The `EVAgentState` TypedDict is the single source of truth passed across all node transitions:

Table 3: Agent state fields

Field	Purpose
<code>raw_data</code>	Uploaded records for selected station
<code>processed_data</code>	Imputed, model-ready feature frame
<code>predictions</code>	Per-record hourly demand forecasts
<code>hotspots</code>	Peak-load windows with severity labels
<code>retrieved_docs</code>	FAISS-retrieved planning passages
<code>reasoning</code>	List of planning rationale strings
<code>llm_reasoning</code>	Raw LLM-generated rationale text
<code>llm_used</code>	Boolean flag: LLM call succeeded
<code>final_plan</code>	Structured infrastructure output
<code>summary</code>	Demand KPIs (avg, peak, hour, risk)
<code>errors / warnings</code>	Deduped failure and warning messages
<code>rag_fallback_used</code>	Whether fallback rules were applied

5.6 LLM Reasoning — Groq Integration

The reasoning node augments rule-based logic with a Groq API call to `llama-3.1-8b-instant`. The prompt is constructed from the demand summary and RAG-retrieved context:

```

1 rag_context = "\n".join(state.get("retrieved_docs", []))[:2000]
2 prompt = f"""
3 You are an EV infrastructure planning expert.
4 Demand Summary:
5 - Peak demand: {peak_demand:.2f} kWh at {peak_hour}:00
6 - Average demand: {avg_demand:.2f} kWh

```

```

7 - Recommended chargers: {chosen_count} ({charger_type})
8 Retrieved Guidelines:
9 {rag_context}
10 Write a planning rationale in 4 bullet points covering:
11 1. Why this charger count is correct
12 2. Grid risk level and mitigation
13 3. Scheduling strategy
14 4. Cost vs. utilization trade-off
15 """
16 response = client.chat.completions.create(
17     model="llama-3.1-8b-instant",
18     messages=[{"role": "user", "content": prompt}],
19     max_tokens=300,
20 )

```

Listing 2: LLM prompt construction in `reasoning_node`

The LLM response is appended to the `reasoning` list and stored separately in `llm_reasoning` for display. If the API key is absent or the call fails, the system degrades gracefully to rule-based reasoning with a visible UI warning — no crash, no silent failure.

Note: Groq’s free tier provides 14,400 requests per day with sub-second latency, making it production-viable at academic scale.

6 Optimisation and Planning Logic

6.1 Charger Profile Selection

Three charger tiers are mapped from peak demand:

Table 4: Charger profile selection thresholds

Peak Demand	Capacity (kWh)	Type	Cost Band
> 25 kWh	12.0	DC Fast Charger (50kW+)	High capex
> 15 kWh	8.0	Hybrid AC/DC Charger	Balanced
≤ 15 kWh	6.0	Level 2 AC Charger	Low capex

6.2 Charger Count Optimisation

The optimal charger count n^* minimises a composite score balancing utilisation accuracy and infrastructure cost:

$$n^* = \arg \min_{n \in \{1, \dots, N_{\max}\}} [\delta(n) + 0.01 n]$$

where $\delta(n) = \max(0, u(n) - u_{\text{high}}) + \max(0, u_{\text{low}} - u(n))$ is the distance of utilisation $u(n) = P_{\text{peak}}/(n \cdot C)$ from the target band $[u_{\text{low}}, u_{\text{high}}] = [0.70, 0.90]$, and $0.01n$ is a cost penalty discouraging oversizing.

6.3 Load Balancing and Scheduling

Grid load balancing is triggered when $P_{\text{peak}} > 25$ kWh. Peak windows are defined as $\{h^* - 1, h^*, h^* + 1\}$ and off-peak windows as the six lowest-demand non-peak hours, ensuring no temporal overlap in scheduling guidance.

7 Implementation Details

7.1 Technology Stack

Table 5: Technology stack

Component	Technology
ML Framework	scikit-learn 1.4, pandas, numpy
Agent Framework	LangGraph 0.1 (StateGraph)
LLM	Groq API — <code>llama-3.1-8b-instant</code>
Vector Store	FAISS (flat L2 index)
UI	Streamlit 1.x
Deployment	Streamlit Community Cloud
Language	Python 3.10+
Serialisation	joblib (model), JSON (output)

7.2 Repository Structure

```
EV-charging-demand-prediction/
|-- agent/           # LangGraph workflow, state, graph builder
|   |-- workflow.py # All 8 node definitions + helper functions
|   |-- graph.py    # Graph builder and run_agent_workflow()
|   \-- state.py    # EVAgentState TypedDict
|-- ml/              # ML inference, evaluation, config
|-- rag/              # FAISS ingest, vectorstore, index files
|-- ui/              # Streamlit dashboard
|-- utils/           # Validation, logging, contracts
```

```

|-- data/          # Raw and processed datasets
|-- models/        # Trained model artifacts (.pkl)
|-- reports/       # LaTeX report assets
|-- notebooks/     # EDA and training notebooks
|-- requirements.txt
\-- README.md

```

7.3 Robustness Design

Table 6: Failure modes and system responses

Failure Mode	System Response
Missing required CSV columns	Validation error with field list; halts gracefully
Missing optional features	Imputed from domain defaults; warning appended
Invalid numeric values	<code>pd.to_numeric</code> coercion; imputation
Model prediction error	<code>PREDICTION_FAILURE_MESSAGE</code> ; safe fallback
FAISS retrieval failure	Fallback rules loaded; <code>rag_fallback_used=True</code>
GROQ_API_KEY absent	Rule-based reasoning; warning shown; no crash
Groq API call exception	Logged; rule-based reasoning continues
Weak demand signal	“Insufficient data for reliable planning”
Full agent workflow exception	Safe structured fallback dictionary returned

8 Results and Observations

8.1 Quantitative ML Performance

The best model (Linear Regression Pipeline) achieves:

- **MAE 4.13 kWh:** on a 0–80 kWh range, this is approximately 5% average relative error — acceptable for coarse capacity planning decisions.
- **RMSE 6.08 kWh:** the MAE–RMSE gap indicates occasional high-demand spikes are harder to forecast, attributable to unobserved external factors (events, weather, pricing changes).
- **R^2 0.69:** 69% of demand variance is explained using only historical session data and engineered temporal features. The remaining 31% would require external signals not available in the dataset.

8.2 Qualitative Planning Output Example

For a station with peak demand of 28 kWh at 18:00:

- **Charger profile:** DC Fast Charger (50kW+)
- **Charger count:** 3 — utilisation 77.8%, within the 70–90% target band
- **Load balancing:** Required (demand exceeds 25 kWh threshold)
- **Peak window:** 17:00, 18:00, 19:00
- **Off-peak window:** 01:00–06:00
- **LLM Insight excerpt:** *Deploying 3 DC Fast Chargers maintains utilisation within the efficiency target band while avoiding transformer overload. Time-of-use pricing during peak hours can shift 10–15% of discretionary charging to off-peak windows, reducing peak stress without adding capital cost.*

This output demonstrates the system’s ability to produce operator-ready, explainable recommendations grounded in both quantitative forecasts and retrieved domain knowledge.

8.3 Agent State Transparency

All six rule-based reasoning steps plus the LLM rationale are exposed in the UI. The `rag_fallback_used` and `llm_used` flags allow evaluators and operators to audit which reasoning mode was active for any given output.

9 UI Explanation — Streamlit Dashboard

The **IntelliCharge360** Streamlit dashboard is structured as a progressive disclosure interface:

Table 7: Streamlit UI sections and purpose

Section	What the User Sees
Hero Banner	System name, tagline, four tech badges
Architecture Tabs	System diagram, workflow text, I/O specification
Model Performance	MAE, RMSE, R^2 metrics with interpretation
CSV Upload	Drag-and-drop with schema validation feedback
Station Selector	Dropdown of available station IDs
Agent Run Button	Primary CTA with progress bar and status widget
Executive Summary	Four metric cards (avg, peak, hour, risk level)
Demand Forecast	Hourly trend line chart for selected station
Hotspot Detection	Sortable dataframe of peak-load windows
Reasoning & LLM	Rule logic bullets + LLM AI Insight callout
Infrastructure Plan	Deployment table, schedule arrows, recommendation cards
Structured Output	Full JSON viewer + download button
Sidebar Help	Step-by-step guide; environment variable checklist

The UI is designed for *three audiences*: (1) evaluators who want a fast technical overview, (2) operators who need actionable planning output, and (3) viva audiences who benefit from transparent step-by-step reasoning.

10 Challenges and Solutions

Challenge 1: State propagation across LangGraph nodes.

Fields `llm_used` and `llm_reasoning` were not declared in `EVAgentState` and not forwarded in `graph.py`'s `_format_result`, causing the LLM badge to show “unavailable” even when the Groq API returned HTTP 200. *Solution*: Added both fields to the TypedDict and the result formatter.

Challenge 2: Chronological split leakage risk.

Random train/test splitting on time-series data would leak future information. *Solution*: Enforced strict chronological 80/20 split.

Challenge 3: FAISS index portability.

The FAISS index must be rebuilt if embeddings change. On Streamlit Cloud, the index cannot be rebuilt at runtime. *Solution*: The pre-built index is committed to the repository; ingest is a local developer utility.

Challenge 4: Deployment stability on free tier.

Streamlit Community Cloud has cold-start latency and memory limits. *Solution*: Model and metrics are cached with `@st.cache_resource`; all heavy objects are loaded once at startup.

Challenge 5: Incomplete CSV inputs from users.

Real-world uploads often lack optional feature columns. *Solution*: `prepare_feature_frame` imputes missing columns with domain-appropriate defaults and surfaces warnings instead of errors.

11 Future Scope

F1. Live data integration: Connect to OCPI-compliant charging network APIs for real-time demand feeds, enabling continuous forecast updates.

F2. Multi-station joint optimisation: Extend the planning node to jointly optimise charger deployment across a network of stations, considering shared grid capacity constraints.

- F3. Reinforcement learning for dynamic scheduling:** Train a policy network to dynamically adjust pricing incentives based on real-time demand, moving beyond static off-peak window recommendations.
- F4. Weather and event features:** Incorporate external signals (temperature, local events, public holidays) to reduce the unexplained 31% demand variance.
- F5. Conditional graph routing:** Use LangGraph’s conditional edge API to route low-confidence predictions directly to a human-review node rather than producing potentially misleading automated recommendations.

12 Conclusion

ChargeSense demonstrates a complete, production-oriented AI system that advances EV infrastructure planning from raw session data to explainable deployment decisions. The Milestone 1 ML pipeline establishes a reliable forecast baseline (MAE 4.13 kWh, R^2 0.69). The Milestone 2 agentic layer transforms those forecasts into structured, grounded, and auditable infrastructure plans through a novel combination of LangGraph stateful orchestration, FAISS-based RAG retrieval, utilisation-aware charger optimisation, and Groq LLM reasoning.

The system’s robustness design — graceful degradation at every failure point, explicit state tracking, and transparent reasoning — makes it suitable for real-world operator decision support. It is entirely free-tier deployable, publicly hosted, and fully documented. The project meets all Milestone 2 rubric requirements and constitutes a portfolio-quality demonstration of applied agentic AI for a pressing sustainability challenge.

References

- International Energy Agency. *Global EV Outlook 2023*. IEA, Paris, 2023. <https://www.iea.org/reports/global-ev-outlook-2023>
- E. Wood, C. Rames, M. Muratori, S. Raghavan, and M. Melaina. *National Plug-In Electric Vehicle Infrastructure Analysis*. Technical Report NREL/TP-5400-69031, National Renewable Energy Laboratory, 2017.
- LangChain Team. *LangGraph: Stateful, Multi-Actor Applications with LLMs*, 2024. <https://github.com/langchain-ai/langgraph>
- J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

- L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Streamlit Inc. *Streamlit Documentation*, 2023. <https://docs.streamlit.io>
- Groq Inc. *Groq API Documentation*, 2024. <https://console.groq.com/docs>
- U.S. Department of Energy. *EV Everywhere: National Workplace Charging Challenge*. Office of Energy Efficiency and Renewable Energy, 2021.